Does Language Matter?
Evaluating LLM-Agent Performance Across Languages

Anonymous ACL submission

Abstract

LLM agents are increasingly deployed to carry out multi-step, tool-using work, yet they are evaluated almost exclusively in English. We ask a simple question with a non-obvious answer: when the *same* task is expressed in a different language, does the agent behave differently, and if so, *where* in its pipeline? We build paired English/Bangla versions of four agent benchmarks—OFFICEBENCH, τ^2-bench, WORKBENCH and THEAGENTCOMPANY—holding the environment, the tool schema and the checkers fixed while varying only the language of the instructions (and, where present, of the simulated user), and we run two open-weight models (31B and 20B parameters). On OFFICEBENCH (300 subtasks per condition) the effect of language is **model-dependent in direction**: the 31B model loses 8.0 points in Bangla (p=0.003, exact McNemar) while the 20B model *gains* 6.7 points (p=0.022), so the gap between the two models shrinks from 21.6 points in English to 7.0 in Bangla. A failure-mode analysis explains the divergence. The 31B model's failures are *content-bound*; the 20B model's are *action-format-bound*—97% of its English failures involve a malformed action—and the language shift reduces the prose leakage that causes them. Across all four benchmarks, only 24 of 163 hand-analysed Bangla-only failures are genuine Bangla agent failures; the rest come from English-only checkers, the Bangla user simulator, translation, the harness, or weaknesses that also occur in English. Removing these benchmark-side confounds on both sides puts every significant gap inside its confidence interval around zero. On OFFICEBENCH, 27% of the 31B model's Bangla failures are answers written in Bangla script and rejected by English keyword checkers; crediting them would raise its Bangla score above its English score. We argue that multilingual agent evaluation requires language-aware verification, and we release code, traces and failure codes.

1 Introduction

Language-model agents now plan over many steps, call tools, and edit real artefacts— spreadsheets, calendars, documents, email (Yao et al., 2023; Liu et al., 2024; Zhou et al., 2024). The benchmarks that measure them, however, are written almost entirely in English (Jimenez et al., 2024; Xie et al., 2024; Mialon et al., 2024). This is a consequential gap. Most of the world does not work in English, and an agent that edits a payroll spreadsheet is supposed to be usable by the person who owns the spreadsheet, in that person's language.

The usual assumption is that moving a task into a lower-resource language degrades performance, because multilingual capability lags English in knowledge and reasoning benchmarks (Ahuja et al., 2023; Shi et al., 2023; Bandarkar et al., 2024; Huang et al., 2023). For *agents* this assumption has not been tested carefully, and it hides a more interesting question. An agentic episode is not one forward pass. It is a loop of reading state, reasoning, emitting a structured action, and consuming the result. Language could in principle perturb any of these stages, and a benchmark score collapses all of them into a single bit of success.

We therefore ask three questions. **RQ1:** Does expressing an equivalent task in a different language change end-to-end agent success, and in which direction? **RQ2:** Where does the change act—on task understanding and content, or on the mechanics of tool interaction? **RQ3:** How much of an apparent cross-lingual gap is a property of the *agent*, and how much is a property of the *evaluation harness*?

To answer these we construct a strictly paired bilingual protocol (Figure 1) over OFFICEBENCH (Wang et al., 2025b), a benchmark of 300 office subtasks requiring one, two or three applications. We translate only the system prompt and the task instruction into Bangla—a language

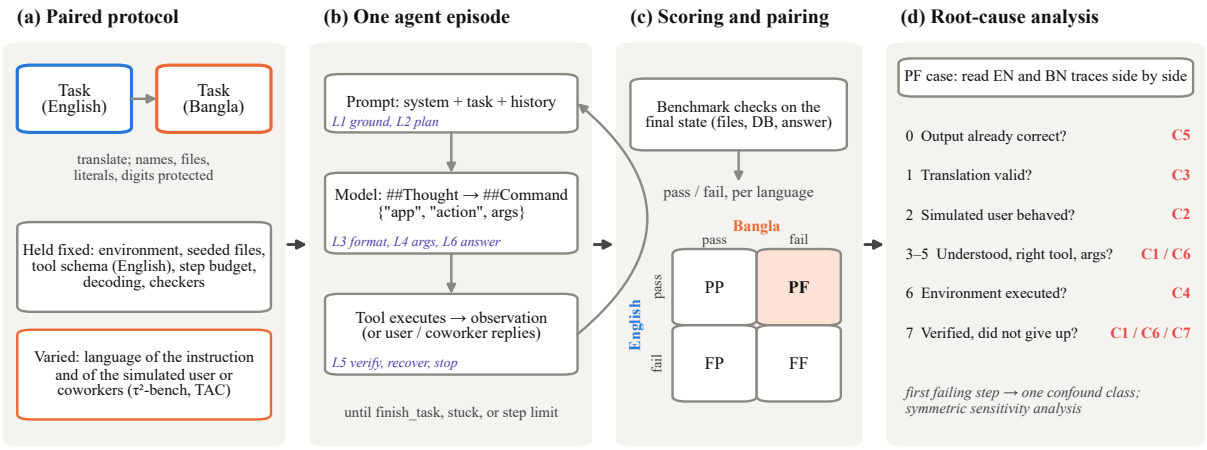


Figure 1: Study overview. (a) Each task runs once in English and once in Bangla; only the language of the instruction (and of the simulated user or coworkers, where a benchmark has them) changes. (b) The agent loop; L1–L6 mark the layers at which a run can first fail. (c) The benchmark's checks score each run, and the two runs of a task form one paired outcome. (d) Every English-pass/Bangla-fail (PF) case is traced to its first failing step and given one confound class C1–C7 (Section 5).

with roughly 240M speakers that is nevertheless under-represented in LLM training data (Joshi et al., 2020)—while holding the Docker environment, the file state, the tool schema and the automatic checkers byte-identical across conditions. We evaluate two open-weight models of comparable but distinct scale and lineage, and we code every failure. We then apply the same paired protocol and failure analysis to three further benchmarks with different interaction designs: customer-service dialogues with a simulated user (τ^2-bench), single-turn function calling against a workplace database (WORKBENCH), and long-horizon company tasks with simulated coworkers (THEAGENTCOMPANY).

Our findings overturn the simple "lower-resource is worse" expectation:

- **Direction is model-dependent.** On OFFICEBENCH, `gemma4:31b` falls from 50.3% to 42.3% in Bangla (p=0.003); `gpt-oss:20b` *rises* from 28.7% to 35.3% (p=0.022). Aggregate "multilingual degradation" is not a property of the language.
- **Model gaps do not transfer.** The English gap between the two models (21.6 points, $p=2\times10^{-11}$) shrinks to 7.0 points in Bangla (p=0.020). A leaderboard built in English overstates the difference a Bangla user would see.
- **Failure mode predicts direction.** The models fail for different reasons. When failures are format-bound, a language shift can help; when they are content-bound, it hurts.
- **Evaluators are part of the measurement.** 27% of the 31B model's Bangla failures are Bangla-script answers rejected by English keyword checks. The measured language penalty is, in large part, a property of the checker.
- **Most Bangla-only failures are not agent failures.** Across four benchmarks, 24 of 163 hand-analysed Bangla-only failures are genuine Bangla agent failures. The dominant confound follows the benchmark design—the checker on OFFICEBENCH, the simulated user on τ^2-bench—and removing confounds on both sides brings every significant gap within noise (Section 5).
- **The effect depends on the type of work.** Grouping the tasks of three further benchmarks into nine task types, failure rates differ far more between types than between languages, and the Bangla penalty concentrates in dialogue tasks in which a Bangla-speaking simulated user takes part (Section 6).

2 Related Work

Agent benchmarks. Interactive benchmarks evaluate LLMs as decision makers rather than text producers: AGENTBENCH (Liu et al., 2024), WEBARENA (Zhou et al., 2024), OSWORLD (Xie et al., 2024), SWE-BENCH (Jimenez et al., 2024) and GAIA (Mialon et al., 2024). OFFICEBENCH (Wang et al., 2025b) is the setting we adopt: it

requires switching between office applications and grounding actions in a real file system, and it scores outcomes with execution-based checks rather than string overlap. τ^2-bench (Barres et al., 2025), WORKBENCH (Styles et al., 2024) and THEAGENTCOMPANY (Xu et al., 2024) extend this to dialogue with a simulated user, workplace databases and long-horizon company tasks; we use them in Section 6. All of these are English-only.

Multilingual evaluation. Cross-lingual evaluation of LLMs is well developed for static tasks—XNLI (Conneau et al., 2018), MEGA (Ahuja et al., 2023), MGSM (Shi et al., 2023), BELEBELE (Bandarkar et al., 2024)—and consistently reports an English advantage, with the gap widening for lower-resource languages (Huang et al., 2023; Zhang et al., 2023). Etxaniz et al. (2024) show that models often perform better when a prompt is first translated into English, implying an English-centric internal pivot. These studies evaluate single-turn outputs; we ask what happens when the output must be a *valid action in a loop*.

Tool use and structured output. Agents must emit well-formed, schema-conforming calls (Schick et al., 2023; Qin et al., 2024; Patil et al., 2024). Format adherence is itself a capability, and it is known to be brittle. Our analysis shows that this brittleness, rather than comprehension, dominates one of our two models and determines the sign of its language effect.

Failure analysis. Rather than reporting a single score, a growing line of work dissects *how* systems fail (Bang et al., 2023). We follow that approach: we code every failed episode and treat the code distribution, not the aggregate, as the object of study.

3 Study Design

3.1 Benchmark and environment

OFFICEBENCH provides 300 subtasks spanning eight applications (calendar, excel, ocr, pdf, shell, word, email, and an in-container `llm` helper). Tasks are stratified by the number of applications required: 93 one-app, 95 two-app and 112 three-app subtasks. Each subtask ships a seeded file system and a set of execution-based checks (cell values, file existence, calendar overlap, keyword containment). A subtask passes only if *all* of its checks pass. Every episode runs in a fresh Docker container reset from a pinned image.

3.2 The paired bilingual protocol

Our central design requirement is that language be the *only* variable. Concretely, across the two conditions we hold fixed: the container image and seeded files; the available applications; the tool schema (action names and argument keys); the step budget; the decoding parameters; and the checkers. We vary:

1. **The system prompt**, authored natively in Bangla, including the role description, the application descriptions and the response-format instruction.
2. **The task instruction**, machine-translated into Bangla and cached so that every model sees an identical string for a given subtask.

Crucially, the **tool schema remains in English**. Action identifiers such as `switch_app` and argument keys such as `file_path` are API symbols that the executor matches literally; translating them would change the environment rather than the language of the task, and would make the two conditions incomparable. We therefore present the schema in English in both conditions and attach a Bangla gloss describing what each tool does. This isolates *instruction language* from *interface language*, and it mirrors deployment reality, where an interface is typically English while the user is not.

3.3 Agent scaffold

We use a ReAct-style loop (Yao et al., 2023). At each step the model receives the system prompt, the task, the history of previous (action, observation) pairs, and the actions available in the current application. It must reply with a short reasoning segment followed by exactly one JSON action:

```
##Thought: <reasoning>
##Command: {"app": ...,
"action": ..., ...}
```

The harness parses the JSON after the last `##Command:` marker, so reasoning may not corrupt the action. An episode terminates when the agent emits `finish_task`, repeats an identical action five times (recorded as *stuck*), or reaches the 20-step budget. We log the full trajectory—prompt, reasoning, action, observation—for every step.

3.4 Models

We evaluate two open-weight models served through the same API with identical decoding (temperature $= 0$, top_p $= 1$): `gemma4:31b`

(32.7B parameters) and `gpt-oss:20b` (20.9B parameters).[1] The two are deliberately different in kind: the former is a conventional instruction-following model, the latter exposes a separate reasoning channel, and this architectural difference turns out to drive our central result. For the reasoning model we fold the reasoning channel into the logged `##Thought` so that both conditions are logged identically.

3.5 Scoring

We score every episode with the benchmark's own checks after repairing three defects in the shipped scorer: email checks now read the episode's own file system (the shipped check reads a path that exists only inside the container); mailbox lookup tries the exact account name before case-insensitive matches among folders that hold mail (the shipped fallback raises an error); and text files are read as UTF-8. No check is relaxed. The repairs raise the pass counts from 132, 103, 80 and 97 to 151, 127, 86 and 106 (gemma EN, gemma BN, gpt-oss EN, gpt-oss BN) and do not change the direction of any result; Appendix A gives both sets.

3.6 Metrics and statistics

Our primary metric is the benchmark's pass rate under the repaired scorer. Because the design is *paired*—the same 300 subtasks in both conditions—we test differences with the exact two-sided McNemar test over discordant pairs, which is the appropriate test for paired binary outcomes (McNemar, 1947). We report discordant counts (b, c) alongside each p-value so that effect size is visible.

3.7 Failure coding

We code every failed episode from its trajectory. Besides a general taxonomy (benchmark defect, wrong content, out of steps, premature finish, task read differently, action format), the Bangla condition requires codes that cannot occur in English, notably *answer language*: the agent produced a semantically reasonable answer in Bangla script which an English keyword check rejects. In addition to coding, we extract fully objective signals for all 1,200 episodes: whether any action was rejected as malformed, whether the episode ended stuck, whether it exhausted the step budget, whether it finished and still failed, episode length, and whether the final answer contained Bangla script.

[1] Served via Ollama; sizes read from the served model metadata.

Model	Lang	1-app (93)	2-app (95)	3-app (112)	All (300)
gemma4:31b	EN	67.7	63.2	25.0	**50.3**
	BN	51.6	53.7	25.0	42.3
gpt-oss:20b	EN	44.1	38.9	7.1	28.7
	BN	53.8	43.2	13.4	35.3

Table 1: OFFICEBENCH pass rate (%) by condition and task complexity (repaired scorer, Section 3.5). Paired exact McNemar: gemma EN vs BN $p = 0.003$ (b=42, c=18); gpt-oss EN vs BN $p = 0.022$ (b=25, c=45).

4 Results

4.1 RQ1: Language changes success, but not in one direction

Table 1 gives the main result. The two models move in *opposite* directions. `gemma4:31b` drops from 50.3% to 42.3%, an 8.0-point loss, with 42 subtasks passing only in English against 18 passing only in Bangla (exact McNemar $p = 0.003$). `gpt-oss:20b` moves the other way, from 28.7% to 35.3%, a 6.7-point gain, with 25 English-only passes against 45 Bangla-only passes ($p = 0.022$).

The aggregate conclusion is therefore not "Bangla is harder". The language manipulation is identical; only the model differs. Any claim of the form "model performance degrades in language L" is under-specified unless the model is named.

Model gaps do not transfer across languages. This has a direct practical consequence. In English, `gemma4:31b` beats `gpt-oss:20b` decisively: 82 subtasks are solved only by gemma against 17 only by gpt-oss ($p = 2.2 \times 10^{-11}$), a gap of 21.6 points. In Bangla the same comparison yields 48 against 27 ($p = 0.020$): the ordering holds, but the gap shrinks by two thirds, to 7.0 points. A practitioner choosing a model from an English leaderboard would expect a far larger advantage than a Bangla user would see.

4.2 RQ2: Complexity gates the language effect

Table 1 also shows that the language effect is not uniform across difficulty. For `gemma4:31b` the Bangla penalty is largest on one-app tasks (-16.1 points) and *exactly zero* on three-app tasks (25.0% in both conditions).

	gemma4:31b		gpt-oss:20b	
Signal	EN	BN	EN	BN
Failures	149	173	214	194
malformed action	49	23	**207**	**177**
ended stuck	3	5	30	14
out of steps	46	39	81	58
finished, failed	99	128	102	121
Mean steps/episode	10.9	10.5	13.3	12.1
Bangla-script answer	0	**46**	0	7

Table 2: Failure composition over all 300 subtasks per condition. The three terminal states (stuck / out of steps / finished-but-failed) are mutually exclusive and sum to the failure count together with one unsaved episode per condition; *malformed action* is a cross-cutting flag counting failures in which at least one action was rejected, so it overlaps the others. gpt-oss is dominated by action-format failures; gemma by episodes that finish and still fail.

The explanation is a floor effect: both models fail most three-app workflows (OCR, summarise, split, send mail) in either language, and where a model has no competence to lose, language cannot take any away. A cross-lingual evaluation run only on hard tasks therefore under-reports the language effect.

4.3 RQ3: Failure mode determines the sign

Why do the two models move in opposite directions under an identical manipulation? Because they fail for different reasons (Table 2).

`gpt-oss:20b` is **format-bound**. 207 of its 214 English failures (97%) contain at least one action rejected as malformed. Its reasoning lives in a separate channel, and in English it frequently spills prose into the action slot, producing a rejected action, then another, until the episode is consumed. Under Bangla instructions this leakage falls: malformed-action failures drop to 177, stuck episodes halve ($30 \rightarrow 14$), budget exhaustion falls ($81 \rightarrow 58$), and episodes shorten ($13.3 \rightarrow 12.1$ steps). The model is not understanding the task better; it is *talking less* into the action channel, and it therefore completes more episodes. Its gain in Bangla is a format effect, not a comprehension effect.

`gemma4:31b` is **content-bound**. It emits valid actions (only 49 of 149 English failures involve a malformed action) and its dominant failure is an episode that runs to completion and is then rejected on content—99 in English rising to 128 in Bangla. This is the mode that a language change should affect, and it does.

This yields a simple predictive rule, and the central claim of this paper:

> *The sign of an agent's cross-lingual effect follows its dominant failure mode. Content-bound agents lose under a language shift; format-bound agents may gain, because the shift suppresses the verbosity that corrupts their actions.*

4.4 RQ4: Much of the measured penalty belongs to the evaluator

The last row of Table 2 isolates a failure mode that *cannot* occur in English. In 46 of its 173 Bangla failures (27%), `gemma4:31b` terminated by calling `finish_task` with an answer written in Bangla script. For 41 of those subtasks a failing check is `evaluate_contain`, which tests whether an English keyword occurs in the output.

The agent was asked in Bangla, answered in Bangla, and was marked wrong because the checker was written in English—a property of the measuring instrument, not of the agent. Under the strict checkers `gemma4:31b` scores 42.3% in Bangla. If every Bangla-script answer were credited—a deliberate upper bound, since we do not verify each one semantically—it would score 57.7%, which is *above* its own English score of 50.3% (for `gpt-oss:20b` the bound moves 35.3% only to 37.7%). Reading the traces, 15 of the 42 subtasks that gemma passes only in English are of this kind: the Bangla answer has the right content (for example "40" written in Bengali digits) and fails only on script; crediting just those 15 lowers the measured gap from 8.0 to 3.0 points. The true value lies between the two bounds, and we cannot determine it with English-only checkers. The honest conclusion is therefore stronger and more useful than the naive one: **we cannot claim from these numbers that Bangla degrades this model's task competence, because the instrument that produced the number is itself language-biased.**

Notably the confound is itself model-dependent: `gpt-oss:20b` answered in Bangla only 7 times, because it remains English-anchored even under Bangla instructions. A checker bias thus penalises precisely the model that adapts to the user's language—exactly the behaviour one would want to reward.

	n	EN	BN	Gap [95% CI]	p
OB gemma	300	50.3	42.3	+8.0 [3.0, 13.0]	.003
OB gpt-oss	300	28.7	35.3	−6.7 [−12.0, −1.3]	.022
τ^2 gemma	218	49.5	38.1	+11.5 [5.0, 17.4]	.001
τ^2 gpt-oss	149	38.9	39.6	−0.7 [−10.7, 8.7]	1.00
WB gemma	90	81.1	82.2	−1.1 [−10.0, 7.8]	1.00
WB gpt-oss	90	58.9	53.3	+5.6 [−5.6, 16.7]	.424
TAC gemma	23	39.1	26.1	+13.0 [0.0, 26.1]	.250

Table 3: Pass rate (%) on the four benchmarks (OB: OFFICEBENCH, WB: WORKBENCH, TAC: THEAGENTCOMPANY, full completion). Gap: English minus Bangla, positive when Bangla is worse; interval from a paired bootstrap over tasks (4,000 resamples); p: exact McNemar.

	OfficeB.		τ^2		WorkB.		TAC	
Confound	g	o	g	o	g	o	g	All
C1 genuine Bangla	6	7	6	2	2	1	0	24
C2 simulated user	0	0	19	16	0	0	0	35
C3 translation	6	0	1	0	1	0	1	9
C4 tool / env.	5	2	0	0	0	0	1	8
C5 checker / task	15	6	0	0	1	0	0	22
C6 also in English	3	1	14	7	3	14	3	45
C7 unclear	7	9	0	1	0	0	3	20
Cases	42	25	40	26	7	15	8	163

Table 4: Confound class of every hand-analysed Bangla-only failure (g: gemma4:31b, o: gpt-oss:20b). TAC cases include Bangla reruns that disagree and both-fail tasks with different partial scores.

5 Four Benchmarks: Results and Root Causes

Additional benchmarks. τ^2-bench (Barres et al., 2025) consists of customer-service dialogues (airline, retail, telecom, banking) in which an LLM-simulated customer talks to the agent; in the Bangla condition the simulated customer also speaks Bangla. We use 218 paired tasks for `gemma4:31b` and 149 for `gpt-oss:20b`, after removing pairs with an infrastructure error in either language. WORKBENCH (Styles et al., 2024) has single-turn workplace tasks (email, calendar, analytics, project management, CRM and multi-domain) scored by comparing the final database with the ground truth; we use a 90-task pilot, 15 per domain. THEAGENTCOMPANY (Xu et al., 2024) has long-horizon company tasks run by an OpenHands agent (Wang et al., 2025a) with LLM-played coworkers, scored by checkpoints; we use 23 task-runs (21 tasks) with `gemma4:31b` only, and count a run as passed only when it earns every checkpoint. The protocol is the one of Section 3.2: the instruction (and, where present, the simulated customer or coworkers) is in Bangla, while tool names, identifiers, file names and URLs stay in English; there is one run per cell at temperature 0.

Root-cause coding. Each English-pass/Bangla-fail case gets its first failure point (the earliest step that, corrected alone, would most likely have passed) and one confound class from the decision tree of Figure 1d (classes in Table 4). Sensitivity analyses then remove confounded tasks, on both sides where English-only failures were coded too.

The language effect depends on model and benchmark. Table 3 shows a significant Bangla penalty for `gemma4:31b` on OFFICEBENCH and τ^2-bench (a second set of 219 τ^2 tasks agrees: $33.8\% \rightarrow 25.1\%$, $p = 0.034$), a significant Bangla *advantage* for `gpt-oss:20b` on OFFICEBENCH, and no detectable gap on WORKBENCH or on the small THEAGENTCOMPANY pilot.

Most Bangla-only failures are not Bangla agent failures. Only 24 of the 163 cases (15%) are genuine Bangla agent failures (Table 4). The dominant confound follows the benchmark's design, that is, which component reads Bangla text: English keyword checkers on OFFICEBENCH (C5), the Bangla simulated customer on τ^2-bench (C2), which often confirms an action and ends the dialogue in the same turn, and language-independent argument errors on WORKBENCH (C6), where the tool descriptions stay in English and no free-text answer is checked. On THEAGENTCOMPANY the only stable English-only pass is again a checker artefact: the coworkers reply in Bangla and a checkpoint looks for the English word "helpful".

Gaps shrink toward zero once confounds are removed. Removing benchmark-side confounds on both sides of the comparison (Figure 2) moves the OFFICEBENCH gaps from +8.0 to +1.9 points (gemma) and from −6.7 to −3.3 (gpt-oss), the τ^2 gemma gap from +11.5 to +5.8, and the THEAGENTCOMPANY gap to zero; every interval then includes zero. This does not show that Bangla has no cost: with one run per cell the remaining genuine cases are too few to establish or exclude a small one.

Two genuine mechanisms recur. The genuine cases share two mechanisms, both seen in both models. First, a name that appears only in Bangla

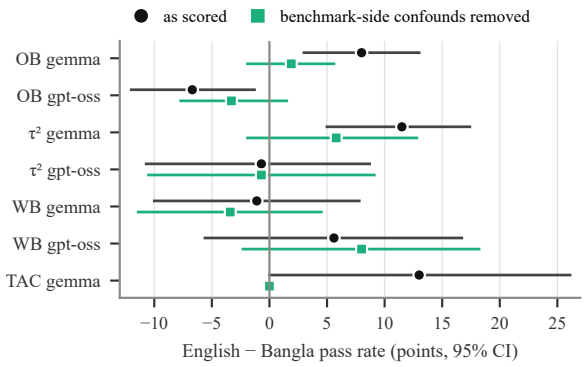


Figure 2: English minus Bangla pass rate as scored and after removing benchmark-side confounds (checker, translation, harness; on τ^2, dialogues the simulated user ended early, in either language; on TAC, reruns that disagree). Every significant gap moves inside an interval containing zero.

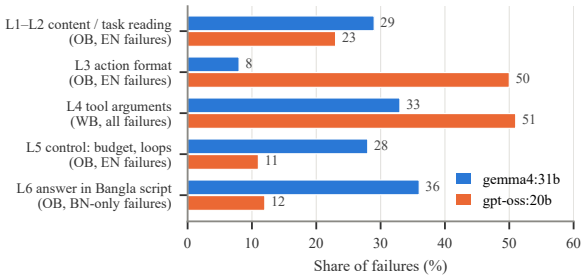


Figure 3: Layer of the agent loop (Figure 1b) at which failures arise, by model. OB: OFFICEBENCH (L1–L3, L5: English failures by primary root cause; L6: share of Bangla-only failures); WB: WORKBENCH, all failed runs.

script has no letter case; the agent writes it in lower case ("bob") for a case-sensitive mailbox or calendar lookup, receives an empty result and accepts it (3 gemma and 6 gpt-oss OFFICEBENCH cases, two of them the same subtasks). Second, the agent renders a value back into a different English form: "Trip" for travelling, "curd" for yogurt, "08:00 AM" for 8:00, or a rolling seven-day window for "last week" on WORKBENCH. No Bangla-only τ^2 failure starts with malformed tool syntax, and no WORKBENCH tool argument contains Bangla script.

The models fail at different layers. The split of Section 4.3 holds beyond OFFICEBENCH (Figure 3). `gpt-oss:20b` fails early: half of its English OFFICEBENCH failures produce no valid action or give up, and on WORKBENCH, where provider-side function calling handles the format, its failures move one layer down to wrong tool arguments. `gemma4:31b` gets past those layers and fails later, on content, step budget and the language of its answer—where the Bangla condition and an English-only checker catch it.

6 Task Taxonomy and Per-Category Results

OFFICEBENCH stratifies tasks only by the number of applications they need (Section 4.2). To ask whether the language effect depends on the *kind* of work, we group the tasks of the three further benchmarks of Section 5 into nine task types and compare failure rates per type, language and model.

Taxonomy. The nine types are listed in Table 5; their definitions, typical failures and example tasks are in Appendix D (Tables 7 and 8). Every task belongs to exactly one type, so the types partition each benchmark.

Type matters more than language. Within every model–benchmark pair, failure rates differ far more between task types than between languages. On WORKBENCH, `gemma4:31b` fails no entity-creation or single-target task in either language but 42% of conditional tasks in both; on τ^2-bench, multi-target operations fail 73–93% of the time in every cell. The types that are hard in English remain the hard ones in Bangla.

The Bangla penalty sits where Bangla reaches a component the agent does not control. The largest increases among types with at least ten tasks are on τ^2-bench: single-target tasks for `gemma4:31b` (35% → 62%, n=34), entity creation for `gpt-oss:20b` (55% → 82%, n=11) and conditional decisions for `gemma4:31b` (27% → 43%, n=37). These are dialogues in which the simulated customer speaks Bangla. In our trace review, 35 of the 66 τ^2 tasks that pass only in English fail because of the Bangla simulated customer—most visibly when it confirms an action and ends the dialogue in the same turn, before the agent can make the required change. On WORKBENCH, where there is no simulated user and tool descriptions stay in English, `gemma4:31b`'s failure count differs by at most two tasks between languages in every type. The one THEAGENTCOMPANY interpersonal task that passes only in English is the checker effect of Section 4.4: the coworkers answer in Bangla, the agent records their replies verbatim, and the checkpoint looks for the English word "helpful".

Task type	WorkBench gemma4:31b EN/BN	Δ	WorkBench gpt-oss:20b EN/BN	Δ	τ^2-bench gemma4:31b EN/BN	Δ	τ^2-bench gpt-oss:20b EN/BN	Δ	TheAgentCompany gemma4:31b EN/BN	Δ
Entity creation	$0/0_{8}$	0	$38/25_{8}$	−13	$67/73_{15}$	+6	$55/82_{11}$	+27	$50/75_{4}$	+25
Single-target	$0/0_{23}$	0	$13/26_{23}$	+13	$35/62_{34}$	+27	$71/74_{34}$	+3	$100/100_{3}$	0
Multi-target and batch	$24/16_{25}$	−8	$56/48_{25}$	−8	$73/80_{44}$	+7	$81/93_{27}$	+12	–	
Temporal reasoning	$40/40_{5}$	0	$60/80_{5}$	+20	–		–		$100/100_{1}$	0
Data analysis and QA	$0/14_{7}$	+14	$14/57_{7}$	+43	$0/100_{1}$	+100	$0/0_{1}$	0	$100/100_{4}$	0
Conditional decisions	$42/42_{19}$	0	$63/68_{19}$	+5	$27/43_{37}$	+16	$37/27_{30}$	−10	$100/100_{2}$	0
Cross-tool composition	$33/33_{3}$	0	$33/33_{3}$	0	–		–		–	
Interpersonal coord.	–		–		–		–		$40/60_{5}$	+20
System setup and troubl.	–		–		$53/59_{87}$	+6	$61/50_{46}$	−11	$0/25_{4}$	+25
All tasks	$19/18_{90}$	−1	$41/47_{90}$	+6	$50/62_{218}$	+11	$61/60_{149}$	−1	$61/74_{23}$	+13

Table 5: Failure rate (%) in English/Bangla per task type; subscript: number of tasks (task-runs for TheAgentCompany); Δ: Bangla minus English in percentage points, so red means Bangla fails more. Rates are descriptive: with fewer than ten tasks one task moves a rate by 10–100 points, and we run no per-type significance tests.

The sign stays model-specific within a type. Conditional decisions on τ^2-bench fail more often in Bangla for `gemma4:31b` (+16) but less often for `gpt-oss:20b` (−10), and the same holds for system setup and troubleshooting (+6 versus −11). The opposite movement of the two models in Section 4.1 therefore persists inside task types and is not an artefact of task mix. On WorkBench, `gpt-oss:20b`'s increases (single-target +13, data analysis +43 on seven tasks) come from a model for which about half of all failures, in both languages, are wrong tool arguments.

7 Discussion

"Multilingual degradation" is not one phenomenon. Our two models received an identical manipulation and moved in opposite directions, with the sign predicted by their failure composition. Reporting a single aggregate delta per language hides this. We recommend that cross-lingual agent evaluations report failure-mode composition alongside the score; it is both more informative and, as we show, necessary to interpret the sign. Model gaps are language-scoped too (21.6 points in English, 7.0 in Bangla), so deployment decisions for non-English users should not be read off English leaderboards.

Evaluation harnesses need language-aware verification. Execution-based checks are usually presented as more robust than surface matching. They are—for file state and cell values. But a large fraction of office tasks are questions whose answer is checked by keyword, and those checks silently encode a language. Multilingual agent benchmarks should either verify answers semantically, accept the task language, or separate "artefact-state" checks from "answer-text" checks and report them apart; the strict and lenient bounds of Section 4.4 are the minimum diagnostic we would want to see reported.

Interface versus instruction language. An agent can be instructed in one language while acting through an English-symbol API; the main cost is not symbol confusion but answer-language mismatch where the agent reports back to the user.

8 Conclusion

We built strictly paired English/Bangla agent evaluations over four benchmarks and two open-weight models. On OfficeBench, task language changes agent success, but the direction depends on the model's dominant failure mode: a content-bound model loses 8.0 points, while a format-bound model gains 6.7. A model gap of 21.6 points in English shrinks to 7.0 in another language. A quarter of one model's Bangla failures are artefacts of English-only checkers, enough to bracket its true Bangla score above its English score. Across four benchmarks, only 24 of 163 Bangla-only failures are genuine Bangla agent failures; the rest lie in the benchmark or recur in English. Language matters for LLM agents—but how it matters is a property of the model and of the evaluator, not of the language alone.

Limitations

One non-English language. We study Bangla only. Bangla is a useful probe—widely spoken

yet under-represented—but our directional claim (sign follows failure mode) should be tested on typologically different and higher-resource languages before being treated as general.

Machine-translated instructions. Task instructions were machine-translated and cached; the system prompt was authored natively. Translation noise is therefore part of the Bangla condition, and some failures we code as content errors may be translation artefacts. We mitigated this by using one fixed translation per subtask for both models, so the comparison *between* models is unaffected, but the absolute Bangla scores should be read as a lower bound on achievable performance.

Two models. Two models cannot establish a general law. They were chosen to differ in kind (conventional versus separate-reasoning-channel), which is what exposes the mechanism, but the mechanism now needs replication across more systems. An evaluation of a third model in the same size class was in progress at submission time and is not reported here.

Decoding is not perfectly reproducible. We decode at temperature 0, but served inference is not bit-deterministic: batching and floating-point non-associativity can change an argmax when two tokens are near-tied, and a single divergence early in an episode can flip the outcome. We did not re-run OFFICEBENCH conditions to estimate this variance directly, so part of the smaller effect we report (the 6.7-point gpt-oss gain, $p = 0.022$) may be run-to-run noise. On THEAGENTCOMPANY, re-running 13 Bangla tasks changed full completion for 1 and the checkpoint score for 3, which shows that this variance is not negligible; a replication study would strengthen the weaker claims. We regard this as the most important missing control.

Failure labels. Each Bangla-only failure carries one confound label assigned by a single annotator per benchmark. A blind second review covered all WORKBENCH cases (Cohen's κ on the confound 0.59 and 0.77 for the two models) and 46 of the 66 τ^2-bench cases ($\kappa = 0.39$ on the fine confound, 0.71 for artefact versus agent); the OFFICEBENCH codebook was checked on 16 double-coded cases ($\kappa = 0.78$). The WORKBENCH and THEAGENTCOMPANY annotators were LLMs (Claude models), and native-speaker review of the translations is pending.

Per-type results are descriptive. Several task types have fewer than ten tasks (Table 5), the three additional benchmarks have one run per cell, and WORKBENCH and THEAGENTCOMPANY are pilot-sized. We therefore report per-type rates without significance tests and draw conclusions only from types with at least ten tasks.

The lenient bound is an upper bound. We did not semantically verify each of the 46 Bangla-script answers; the bound credits them all. The true Bangla score lies strictly between the two bounds and we do not claim to have located it.

Infrastructure effects. Episodes ran on a memory-constrained host. A small number of episodes (at most one per condition in the reported runs) failed to persist their output and are counted as failures, and a per-episode wall-clock cap occasionally terminated a slow episode. These affect all conditions approximately equally and are too few to alter any reported comparison, but they are not zero.

Ethics Statement

This work evaluates publicly available open-weight models on a public benchmark in a sandboxed container; no human subjects and no private data are involved. Our main finding has a direct equity dimension: evaluation harnesses written in English can penalise an agent for answering in the user's language, which systematically understates agent quality for non-English users and rewards models that ignore the user's language. We release our bilingual protocol, trajectories and failure codes so that this bias can be measured rather than inherited. The translated task set was produced by machine translation and may contain errors; it is intended for evaluation research, not as a curated Bangla language resource.

References

Kabir Ahuja, Harshita Diddee, Rishav Hada, Millicent Ochieng, Krithika Ramesh, Prachi Jain, Akshay Nambi, Tanuja Ganu, Sameer Segal, Maxamed Axmed, Kalika Bali, and Sunayana Sitaram. 2023. MEGA: Multilingual evaluation of generative AI. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa,

Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. 2024. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, Quyet V. Do, Yan Xu, and Pascale Fung. 2023. A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. In *Proceedings of the 13th International Joint Conference on Natural Language Processing (IJCNLP-AACL)*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Alexis Conneau, Ruty Rinott, Guillaume Lample, Adina Williams, Samuel R. Bowman, Holger Schwenk, and Veselin Stoyanov. 2018. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Julen Etxaniz, Gorka Azkune, Aitor Soroa, Oier Lopez de Lacalle, and Mikel Artetxe. 2024. Do multilingual language models think better in English? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Haoyang Huang, Tianyi Tang, Dongdong Zhang, Wayne Xin Zhao, Ting Song, Yan Xia, and Furu Wei. 2023. Not all languages are created equal in LLMs: Improving multilingual capability by cross-lingual-thought prompting. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.

Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. 2020. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. 2023. Language models are multilingual chain-of-thought reasoners. In *International Conference on Learning Representations (ICLR)*.

Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. 2024. WorkBench: a benchmark dataset for agents in a realistic workplace setting. In *Conference on Language Modeling (COLM)*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, et al. 2025a. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2025b. OfficeBench: Benchmarking language agents across multiple applications for office automation. In *Findings of the Association for Computational Linguistics: NAACL 2025*. ArXiv:2407.19056.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Lu, Kai Tan, Yanbo Shen, Zhennan Liu, Yuchen Guo, Zhiwen Wang, Yiheng Qi, et al. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, et al. 2024. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Xiang Zhang, Senyu Li, Bradley Hauer, Ning Shi, and Grzegorz Kondrak. 2023. Don't trust ChatGPT when your question is not in english: A study of multilingual abilities and types of LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*.

A Reproducibility details

Each episode runs in a fresh container reset from a pinned image, with a 20-step budget and a single seeded file system per subtask. Decoding is fixed at $\mathrm{temperature} = 0$, $\mathrm{top_p} = 1$. The harness records, per step, the full prompt, the model reply, the parsed action and the environment observation; all analyses in this paper are computed from those logs and from the benchmark's own scorer output, not from intermediate summaries.

Shipped versus repaired scorer. Pass counts out of 300 with the shipped scorer and with the repaired scorer of Section 3.5: `gemma4:31b` English 132 / 151, Bangla 103 / 127; `gpt-oss:20b` English 80 / 86, Bangla 97 / 106. With the shipped scorer the language effects have the same direction (gemma -9.7 points, $p < 0.001$; gpt-oss $+5.6$, $p = 0.043$). The repairs never turn a pass into a fail.

B Failure propagation case studies

Figure 4 traces four Bangla runs from their first divergence from the English run to the final failure. In three of them a small task-level slip specific to the Bangla input (a lower-cased name, a whole-document rewrite, a relative-date window) becomes fatal because the agent does not verify or adapt; in the fourth the agent is correct and the simulated user ends the dialogue.

C Failure code book

Code	Meaning
BM	Benchmark defect: a check or reference asks for something the task does not state
WC	Wrong content: output exists, a value is wrong
SB	Out of steps: hit the 20-step budget
PF	Premature finish: declared done with a required output missing
TR	Task read differently: a different operation from the one asked
FMT	Action format: could not produce a valid action
ER	Empty result taken as "nothing exists"
Bangla-condition only	
AL	Answer language: correct answer in Bangla script, English check fails
NC	Name case: a name rewritten from Bangla script with wrong case
WD	Word choice when translating a literal back to English
TX	Translation changed the task or a literal

Table 6: Failure codes. AL–TX cannot occur in the English condition and are the language-specific modes discussed in Section 4.4.

D Task-type examples

Table 7 defines the nine task types of Section 6 and their typical failures; Table 8 gives example tasks for each type. WORKBENCH prompts are quoted from the task templates (message bodies shortened); τ^2-bench scenarios are paraphrased; for THEAGENTCOMPANY we give a summary of the task and the checkpoints passed in each language.

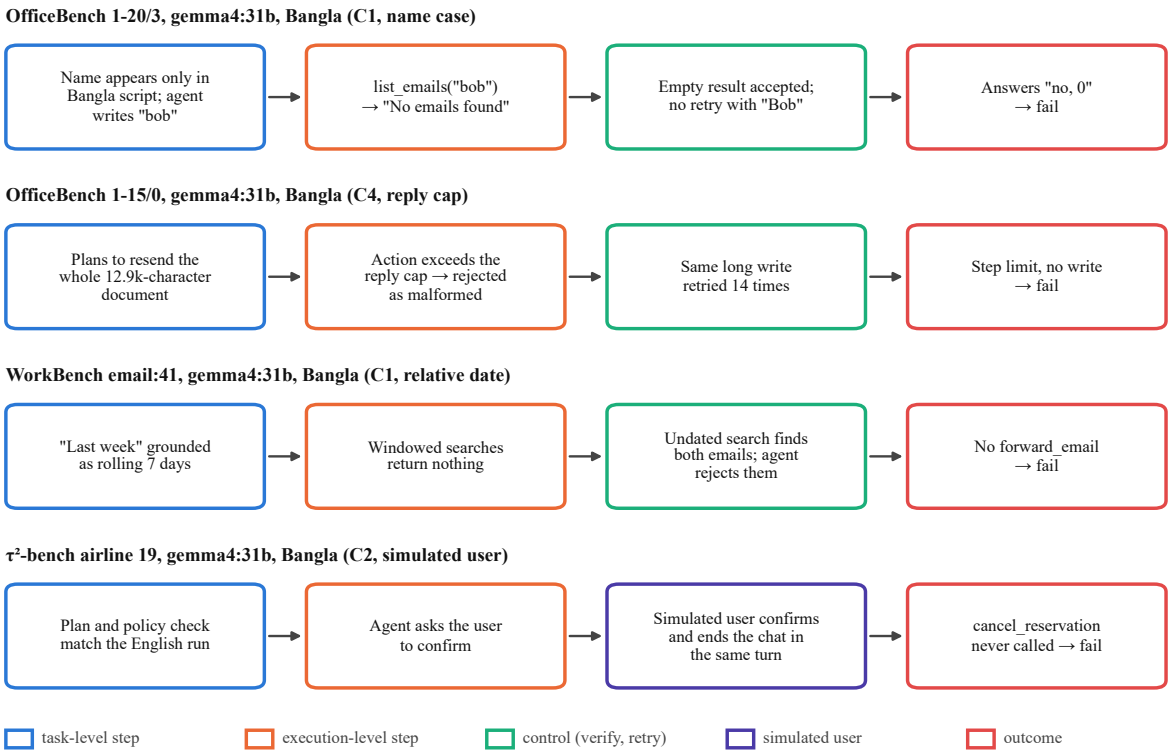


Figure 4: Four traced failure chains (Bangla runs whose English counterpart passes). Box colour marks the kind of step; the confound class is given in parentheses.

Task type	What the task asks	Typical failure (from trace review)
Entity creation	Make one new record from details given in the request (send an email, book a flight, open an account, write a document)	Wrong field values; on τ^2 the wrong product or a skipped eligibility check; on TheAgentCompany an incomplete document
Single-target retrieval and action	Find the one record the request points to, then change it (delete, update, return, move)	Wrong record or argument (an ID sent as a name), or stops before the last step; on τ^2 in Bangla often triggered by the simulated customer
Multi-target and batch operations	Select every record that matches conditions, or make several changes in one request	Does part of the job, misses a matching item, or ends the conversation early
Temporal reasoning	Work out free slots, relative dates or time shifts	Wrong slot, "due today" counted as overdue, or a Bangla date phrase misread
Data analysis and QA	Compute numbers from data, chart them, or look something up and answer	Wrong metric or number, file not reachable (TheAgentCompany), result not checked
Conditional decision-making	Check a condition or policy first; act only if it holds, otherwise refuse, do nothing or escalate	Acts before checking, gives in to a pushy customer, or does not escalate
Cross-tool composition	Take a value from one tool and use it as input to another	Passes the wrong value from the first tool to the second
Interpersonal coordination	Talk to coworkers in chat to obtain information or agreement	Stops after the first reply and never follows up with a coworker who did not answer
System setup and troubleshooting	Install software, or walk a user through fixing a device	Skips the final check, uses the wrong tool, or gives up when several faults are present

Table 7: The nine task types.

Task type	Examples
Entity creation	*WorkBench email:20*: "I need to send an email to kofi saying '…' and title it 'Task Update on Improve UX of onboarding process'. Can you do that?" τ^2 *airline 8*: book a one-way flight from ORD to PHL on May 26 with an extra passenger. *TheAgentCompany hr-new-grad-job-description*: write a new-grad job description from a template and a requirements file and save it on the cloud drive (EN 3/3, BN 1/3).
Single-target	*WorkBench email:6*: "Delete my last email from chenwei." τ^2 *airline 16*: change an upcoming ATL–PHL trip to the cheapest Economy (not Basic Economy) flight on the next day. *TheAgentCompany admin-remove-pages-pdf*: download a PDF from the cloud drive, remove pages and save the result (EN 1/3, BN 0/3).
Multi-target and batch	*WorkBench email:41*: forward all emails from anaya sent last week about 'Update on Board of Directors Conclave' to nadia. *WorkBench crm:77*: move all customers who have not responded to a software proposal in 6 weeks to lost. τ^2 *airline 22*: change the passenger, upgrade to economy and add 3 checked bags in one request.
Temporal reasoning	*WorkBench multi_domain:84*: if olga has overdue tasks, book a half-hour meeting 'Catch up on overdue tasks' at the earliest time the user is free tomorrow. *TheAgentCompany admin-arrange-meeting-rooms*: find the minimum number of meeting rooms for six teams' overlapping meetings (EN 0/2, BN 0/2).
Data analysis and QA	*WorkBench analytics:20*: plot the distribution of engaged users and total visits between November 5 and November 21. τ^2 *airline 3*: tell the customer how many suitcases the reservation allows (the agent must verify that the customer is Silver, not Gold). *TheAgentCompany ds-answer-numerical-data-question*: answer numeric questions about a spreadsheet (EN 0/6, BN 0/6).
Conditional decisions	*WorkBench analytics:119*: make bar charts of social media and search engine traffic over the last 4 weeks if social media brought more traffic. τ^2 *airline 26*: refuse a cancellation with refund when the policy criteria are not met. *TheAgentCompany qa-escalate-emergency*: report a security bug and escalate to the manager if the developer does not reply (EN 2/3, BN 2/3).
Cross-tool composition	*WorkBench multi_domain:25*: find the email from November 20 about 'Task Update on Implement payment gateway API' and schedule a 30-minute meeting with its sender on December 18 at 12:30.
Interpersonal coord.	*TheAgentCompany admin-ask-for-meeting-feedback*: ask coworkers for feedback on a meeting and record it in a spreadsheet (EN 6/6, BN 4/6).
System setup and troubl.	*TheAgentCompany sde-install-go*: install the Go toolchain and put it on PATH (EN 2/2, BN 0/2 in one of two task sets). τ^2 *telecom-11*: the customer cannot send MMS; three hidden faults (bad network preference, data mode off, roaming disabled while abroad).

Table 8: Example tasks per type. Checkpoint counts for TheAgentCompany are given as passed/total.